



ITM Business
School
ITM SKILLS UNIVERSITY



ITM SKILLS UNIVERSITY

SCHOOL OF FUTURETECH

B.Tech Computer Science Engineering and AI
Database Management Systems

CASE STUDY / PROJECT REPORT



VoteSecure

Election & Voter Management System

A Relational Database Case Study using PostgreSQL

SUBMITTED BY

Student Name: Tejas Chavan

Student ID: 150096725185

Semester: III

Academic Year: 2025–26

Case Study: No. 87

1. Student Details, Problem Statement & Objectives

1.1 Student Details

Student Name	Tejas Chavan	Student ID	150096725185
Programme	B.Tech Computer Science Engineering and AI	School	School of FutureTech, ITM Skills University
Course	Database Management Systems (DBMS)	Semester / Year	Semester III / 2025–26
Case Study	VoteSecure – Election & Voter Management System (No. 87)	Database	PostgreSQL (local), run through VS Code

1.2 Problem Statement

VoteSecure needs a relational database for a **student council election** that guarantees every registered voter can cast **exactly one vote per election** and produces the **winning candidate of each constituency** the moment polling closes.

Manual or spreadsheet-based elections are slow and error-prone. A database-driven design has to solve four problems:

- **Redundancy:** party name and symbol must be stored once, not repeated on every candidate row.
- **Accuracy:** every vote must be linked to a valid voter, candidate, constituency and election.
- **Fairness:** one voter must never be able to vote twice in the same election.
- **Speed:** constituency winners must be calculated by SQL, without manual counting.

1.3 Objectives

No.	Objective	Section	No.	Objective	Section
1	Design an ER diagram	§3	8	Prevent duplicate voting	§5, §7
2	Identify entities and relationships	§3	9	Use JOINS	§6
3	Convert the ER model to a relational schema	§4	10	Use GROUP BY	§6, §7
4	Normalize the database (separate Party)	§4	11	Use COUNT and other aggregates	§6
5	Create tables using DDL	§5	12	Use a MAX subquery	§7
6	Insert sample data using DML	§5	13	Determine constituency-wise winners	§7
7	Apply PK, FK, UNIQUE and NOT NULL constraints	§5	14	Maintain referential integrity	§4, §5

1.4 Expected Outcomes

- Double voting is impossible: the UNIQUE constraint blocks a second Vote row for the same voter and election.
- Constituency winners are available the instant polling closes, through the MAX-subquery ranking query.
- Candidate-wise vote share is reported for transparency using aggregate percentages.
- Referential integrity ties every vote to a valid voter and candidate.

1.5 Report Contents

2. System Architecture & Workflow	3	7. Winner Determination & Duplicate Vote	8
3. Database Design & ER Diagram	4	8. Execution & Screenshots	9
4. Relational Schema & Normalization	5	9. Results & Charts	10
5. SQL Implementation & Constraints	6	10. Real-World Application & Future Scope	11
6. SQL Queries, JOINS & Aggregates	7	11. Conclusion	12

2. System Architecture & Workflow

2.1 Workflow

VoteSecure keeps all election data in one relational database and lets the database enforce the rules. A vote is stored only after it passes validation, and results are derived with SQL, never typed by hand (Figure 1).

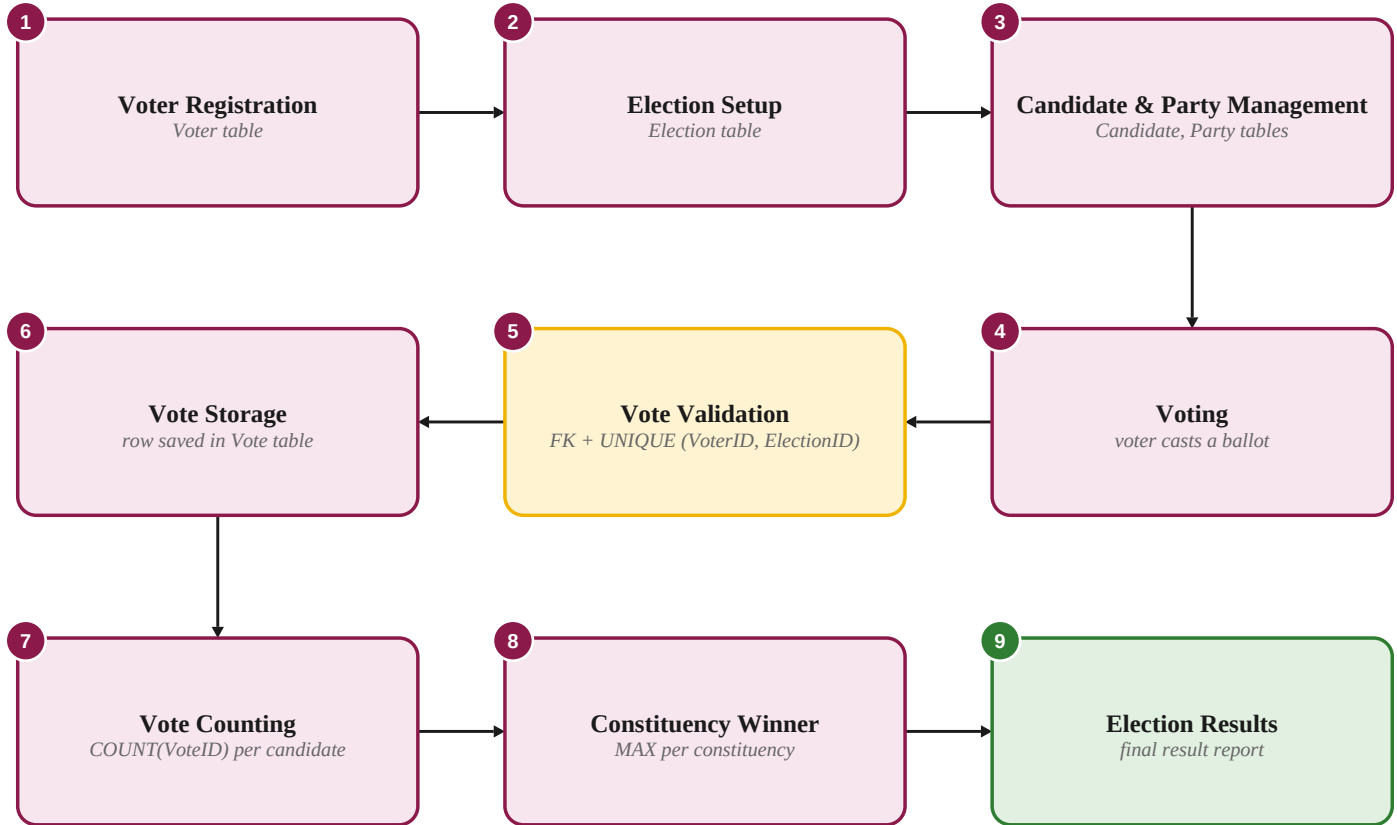


Figure 1: Workflow from voter registration to election results

2.2 Architecture

The implementation is database-focused: scripts are written in VS Code, run on a local PostgreSQL server, and the SQL queries produce the reports (Figure 2). No frontend or backend application is part of this project.

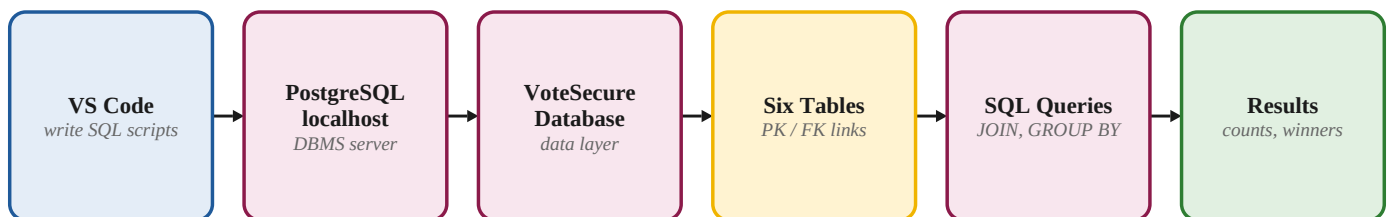


Figure 2: Database architecture of VoteSecure

Layer	Technology	Role
Database	PostgreSQL	Stores data; enforces keys and constraints
Language	SQL (DDL, DML, DQL)	Creates tables, inserts data, queries results
Editor	Visual Studio Code	Used to write and run the SQL scripts
Connection	Localhost	Local PostgreSQL server on the development machine

3. Database Design & ER Diagram

The database has six entities. Each becomes a table whose primary key is an ID column; foreign keys connect related tables.

Entity	Description	Primary key	Foreign keys
Voter	Registered student eligible to vote in one constituency	VoterID	ConstituencyID
Candidate	Student contesting in one constituency for one party	CandidateID	PartyID, ConstituencyID
Party	Party or group with a name and a symbol (stored once)	PartyID	—
Constituency	Electoral division of voters and candidates	ConstituencyID	—
Election	Election event with a name, date and status	ElectionID	—
Vote	One ballot linking voter, candidate, constituency, election and timestamp	VoteID	VoterID, CandidateID, ConstituencyID, ElectionID

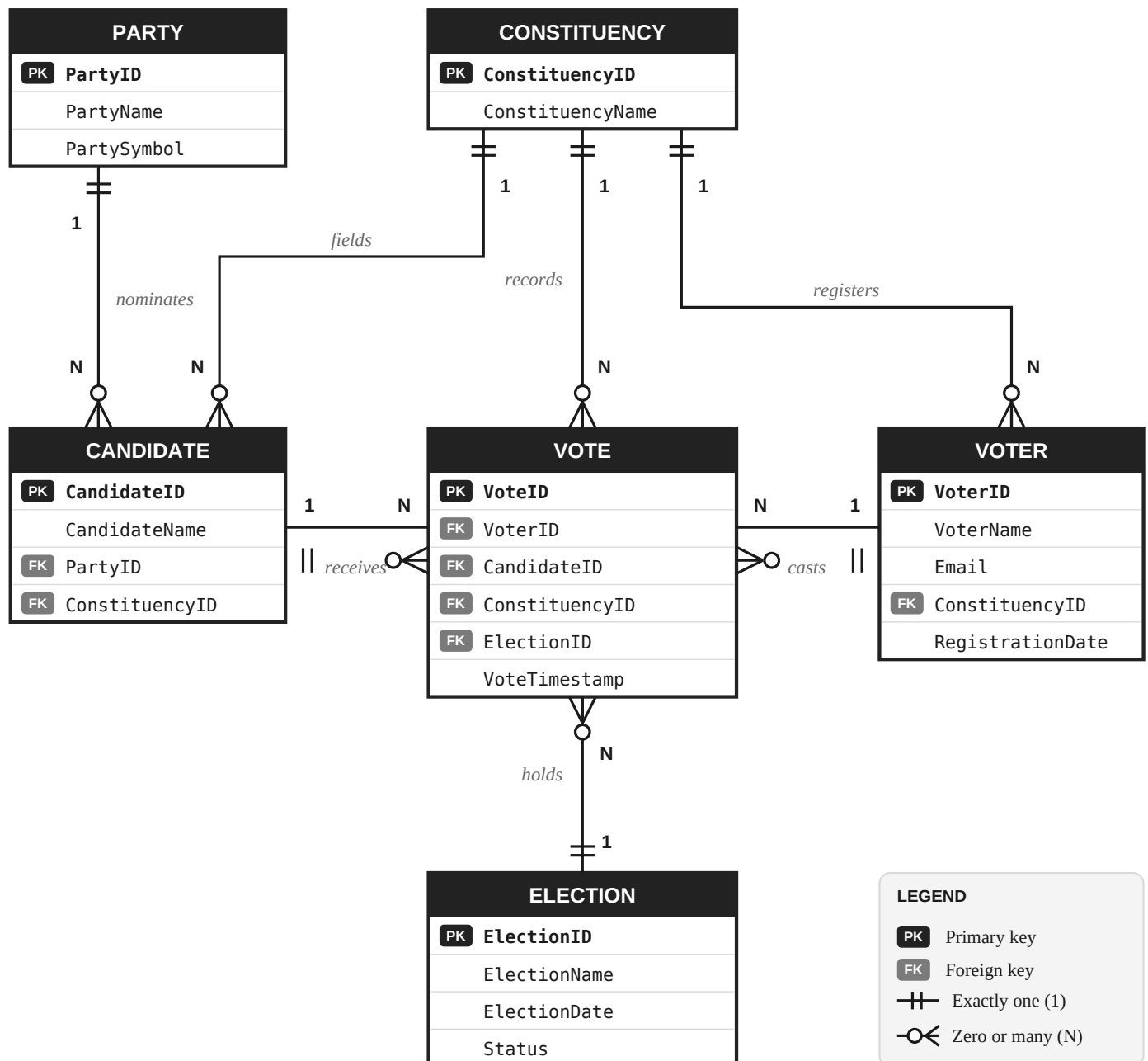


Figure 3: Entity Relationship Diagram of VoteSecure (PK = primary key, FK = foreign key)

Cardinality: all relationships are one-to-many (1 : N): the “1” side is the parent, the “N” side holds the foreign key. Vote is the junction entity resolving the many-to-many link between Voter and Candidate.

4. Relational Schema & Normalization

4.1 Relational Schema

The ER model converts to the schema below. Primary keys are underlined; foreign keys are *italic*.

Relation	Attributes
PARTY	(<u>PartyID</u> , PartyName, PartySymbol)
CONSTITUENCY	(<u>ConstituencyID</u> , ConstituencyName)
VOTER	(<u>VoterID</u> , VoterName, Email, <i>ConstituencyID</i> , RegistrationDate)
CANDIDATE	(<u>CandidateID</u> , CandidateName, <i>PartyID</i> , <i>ConstituencyID</i>)
ELECTION	(<u>ElectionID</u> , ElectionName, ElectionDate, Status)
VOTE	(<u>VoteID</u> , <i>VoterID</i> , <i>CandidateID</i> , <i>ConstituencyID</i> , <i>ElectionID</i> , VoteTimestamp)

Foreign key	References	Foreign key	References
Candidate.PartyID	Party.PartyID	Vote.VoterID	Voter.VoterID
Candidate.ConstituencyID	Constituency.ConstituencyID	Vote.CandidateID	Candidate.CandidateID
Voter.ConstituencyID	Constituency.ConstituencyID	Vote.ConstituencyID	Constituency.ConstituencyID
		Vote.ElectionID	Election.ElectionID

Every foreign key (all 1 : N) refers to an existing primary key, so a vote can never point to a voter, candidate, constituency or election that does not exist: this is **referential integrity**.

4.2 Normalization

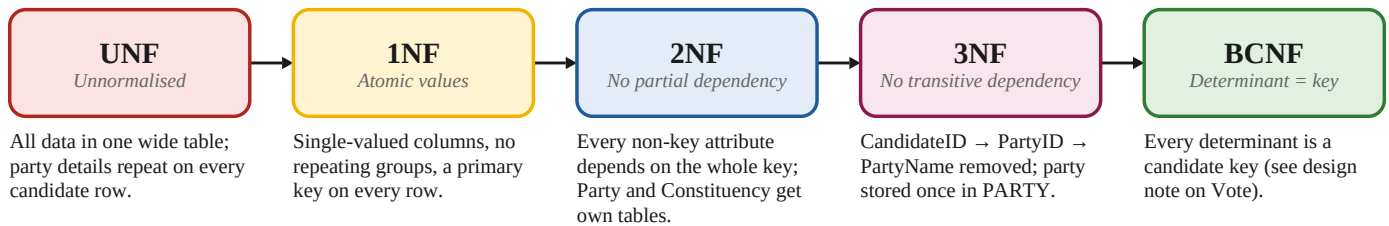


Figure 4: Normalization path from UNF to BCNF

4.3 Why Party Was Separated from Candidate

In a single CANDIDATE table, PartyName and PartySymbol repeat for every candidate of the same party, and they depend on the party rather than on the candidate (CandidateID → PartyID → PartyName). Moving them into PARTY and keeping only the foreign key PartyID in CANDIDATE removes the redundancy: a party is renamed in one row, cannot be stored inconsistently, and can exist before it has candidates (Figure 5). Vote.ConstituencyID can also be derived through Candidate; it is kept in Vote deliberately so constituency totals need no extra join, and every other table satisfies BCNF.

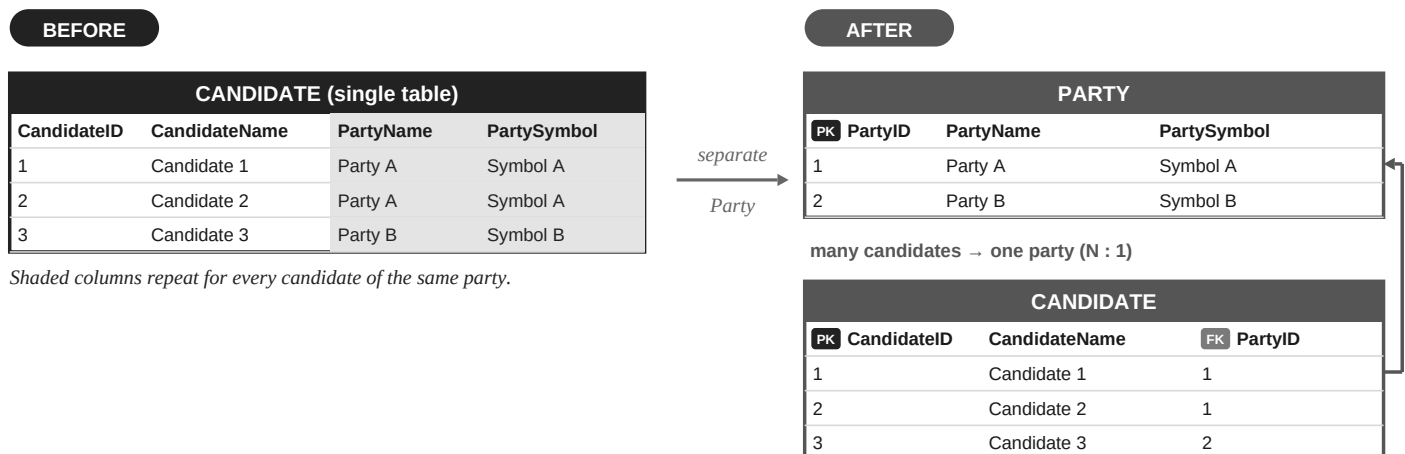


Figure 5: Candidate table before and after separating Party (illustrative rows)

5. SQL Implementation & Constraints

5.1 SQL Command Categories

Category	Commands	Use in VoteSecure
DDL – Data Definition	CREATE, ALTER, DROP	Create the six tables with keys and constraints
DML – Data Manipulation	INSERT, UPDATE, DELETE	Insert sample voters, candidates, parties, votes
DQL – Data Query	SELECT	Joins, counts, vote share and winner reports

5.2 Table Creation (DDL)

Tables are created in dependency order (Party, Constituency, Election, Voter, Candidate, Vote) so a referenced table always exists first; ID columns use SERIAL (auto-increment). The Vote table is the most important definition; the other five follow the same pattern (see schema.sql in Figure 12). The script declares foreign keys as named constraints with ON DELETE RESTRICT ON UPDATE CASCADE; the inline form is shown here for brevity.

```
CREATE TABLE Vote (
  VoteID          SERIAL PRIMARY KEY,
  VoterID         INT NOT NULL REFERENCES Voter (VoterID),
  CandidateID     INT NOT NULL REFERENCES Candidate (CandidateID),
  ConstituencyID  INT NOT NULL REFERENCES Constituency (ConstituencyID),
  ElectionID      INT NOT NULL REFERENCES Election (ElectionID),
  VoteTimestamp   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT uq_voter_election UNIQUE (VoterID, ElectionID)
);
```

UNIQUE (VoterID, ElectionID) allows each (voter, election) pair to appear only once. It **prevents a voter from voting more than once in the same election**, yet the same voter can still vote in a different election. Because the database performs the check, duplicate voting is blocked however the INSERT is issued.

5.3 Sample Data (DML)

Rows are inserted parent-first (Party, Constituency, Election → Voter → Candidate → Vote). Illustrative statements; the actual rows are in seed.sql:

```
INSERT INTO Party (PartyName, PartySymbol) VALUES ('Party A', 'Symbol A');
INSERT INTO Vote (VoterID, CandidateID, ConstituencyID, ElectionID) VALUES (1, 2, 1, 1);
UPDATE Election SET Status = 'Completed' WHERE ElectionID = 1;
```

5.4 Constraints & Data Integrity

Constraint	Purpose	Used in VoteSecure
PRIMARY KEY	Identifies each row uniquely; never NULL	An ID column in every table
FOREIGN KEY	Links a column to a parent table's key	Voter, Candidate and Vote reference their parents (referential integrity)
NOT NULL	Column must always have a value	Names, dates and all foreign key columns
UNIQUE	No two rows share the same value(s)	Voter.Email and UNIQUE (VoterID, ElectionID) on Vote
CHECK	Restricts a column to allowed values	Election.Status IN ('Scheduled', 'Ongoing', 'Completed', 'Cancelled')
DEFAULT	Fills a value when none is given	RegistrationDate, VoteTimestamp

5.5 Referential Integrity

The foreign keys of the Vote table guarantee that every vote refers to an existing voter, candidate, constituency and election. A vote cannot be stored for an unregistered voter, and a parent row cannot be deleted while votes still refer to it.

6. SQL Queries, JOINS & Aggregate Functions

6.1 JOINS

A JOIN combines rows of two tables on a matching column. Figure 6 compares the four types with Candidate (A) and Vote (B).

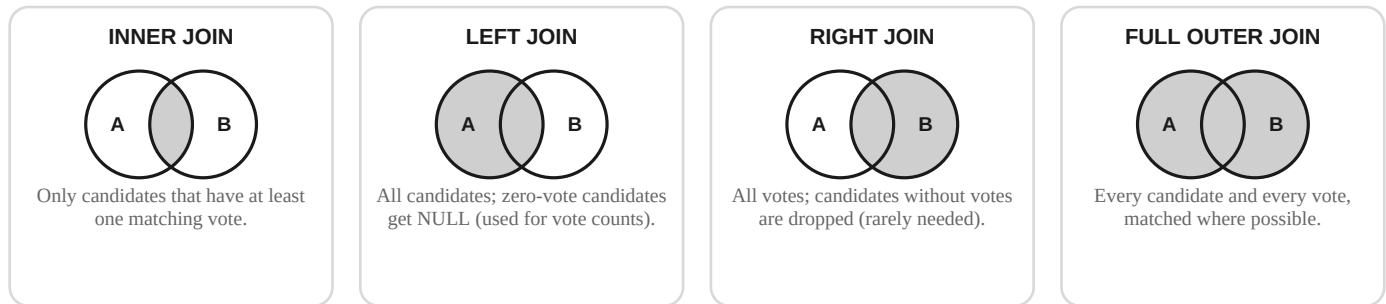


Figure 6: INNER, LEFT, RIGHT and FULL OUTER JOIN

6.2 Aggregate Functions

Function	Meaning	Use in VoteSecure
COUNT()	Number of rows	COUNT(VoteID) turns stored votes into a vote total per candidate or constituency
SUM() / AVG()	Total / average	Grand total and average votes per candidate
MIN()	Smallest value	Lowest vote count among candidates
MAX()	Largest value	Highest vote count; selects the constituency winner

6.3 Representative Queries

Q1. JOIN – candidates with party and constituency

```
SELECT cand.CandidateName, p.PartyName, c.ConstituencyName
FROM Candidate cand
JOIN Party p ON cand.PartyID = p.PartyID
JOIN Constituency c ON cand.ConstituencyID = c.ConstituencyID;
```

Q2. GROUP BY + COUNT – votes per candidate (LEFT JOIN keeps zero-vote candidates)

```
SELECT cand.CandidateName, COUNT(v.VoteID) AS VoteCount
FROM Candidate cand LEFT JOIN Vote v ON v.CandidateID = cand.CandidateID
GROUP BY cand.CandidateID, cand.CandidateName
ORDER BY VoteCount DESC;
```

Q3. Constituency-wise vote count

```
SELECT c.ConstituencyName, COUNT(v.VoteID) AS TotalVotes
FROM Constituency c LEFT JOIN Vote v ON v.ConstituencyID = c.ConstituencyID
GROUP BY c.ConstituencyID, c.ConstituencyName ORDER BY TotalVotes DESC;
```

Q4. Aggregates – MIN, MAX, AVG and SUM of the candidate vote counts

```
SELECT MIN(vc) AS MinVotes, MAX(vc) AS MaxVotes, ROUND(AVG(vc), 2) AS AvgVotes, SUM(vc) AS TotalVotes
FROM (SELECT COUNT(v.VoteID) AS vc
FROM Candidate cand LEFT JOIN Vote v ON v.CandidateID = cand.CandidateID
GROUP BY cand.CandidateID) t;
```

Q5. Candidate vote share within the constituency (%)

```
SELECT cand.CandidateName, c.ConstituencyName, COUNT(v.VoteID) AS VoteCount,
ROUND(100.0 * COUNT(v.VoteID) / NULLIF((SELECT COUNT(*) FROM Vote t
WHERE t.ConstituencyID = cand.ConstituencyID), 0), 2) AS VoteSharePct
FROM Candidate cand
JOIN Constituency c ON cand.ConstituencyID = c.ConstituencyID
LEFT JOIN Vote v ON v.CandidateID = cand.CandidateID
GROUP BY cand.CandidateID, cand.CandidateName, cand.ConstituencyID, c.ConstituencyName
ORDER BY c.ConstituencyName, VoteCount DESC;
```

Q6. Constituency-wise winner (JOIN + GROUP BY + COUNT + MAX subquery) is explained in full in Section 7. The complete set of queries is kept in query.sql.

7. Winner Determination & Duplicate Vote Prevention

7.1 Winner Determination

The winner of a constituency is the candidate with the most votes in it: votes are joined, grouped, counted and compared with the constituency maximum (Figure 7).

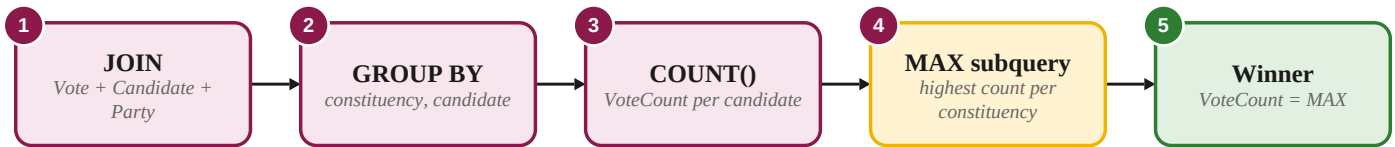


Figure 7: Winner determination (JOIN, GROUP BY, COUNT, MAX subquery)

```

SELECT con.ConstituencyName, c.CandidateName, p.PartyName, COUNT(v.VoteID) AS WinningVotes
FROM Candidate c
JOIN Constituency con ON c.ConstituencyID = con.ConstituencyID
JOIN Party p ON c.PartyID = p.PartyID
JOIN Vote v ON c.CandidateID = v.CandidateID
WHERE v.ElectionID = 1
GROUP BY c.ConstituencyID, con.ConstituencyName, c.CandidateID, c.CandidateName, p.PartyName
HAVING COUNT(v.VoteID) = (
    SELECT MAX(sub.VoteCount)
    FROM (
        SELECT COUNT(v2.VoteID) AS VoteCount
        FROM Candidate c2
        JOIN Vote v2 ON c2.CandidateID = v2.CandidateID
        WHERE c2.ConstituencyID = c.ConstituencyID AND v2.ElectionID = 1
        GROUP BY c2.CandidateID
    ) AS sub
)
ORDER BY con.ConstituencyName;
  
```

- The inner query counts votes per candidate and takes **MAX** only for the candidate's own constituency; **HAVING** keeps the candidates whose count equals it. The query is limited to Election 1; tied candidates would all be returned. Output: Figure 14.

7.2 Duplicate Vote Prevention

The same pair (VoterID, ElectionID) is inserted twice. The first insert is accepted; the second is rejected by **UNIQUE (VoterID, ElectionID)** (Figure 8). In transaction.sql the resulting unique_violation is caught and reported as "Blocked: Voter 1 has already voted in Election 1!" (Figure 9).

FIRST ATTEMPT



SECOND ATTEMPT (same voter, same election)

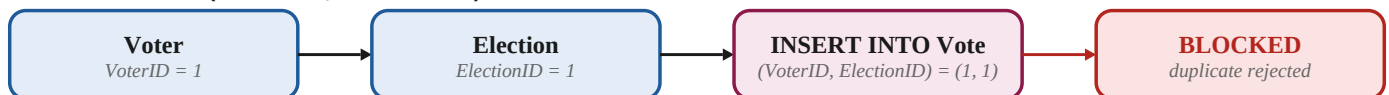


Figure 8: First attempt allowed, second attempt blocked by UNIQUE (VoterID, ElectionID)

```

33 EXCEPTION
34 WHEN unique_violation THEN
35 RAISE NOTICE 'Blocked: Voter 1 has already voted in Election 1!';
36 END $$;
37
  
```

MyBatis

Problems Output Debug Console Terminal Ports Postman Console PostgreSQL Query Results MyBatis

Messages

localhost | postgres | schema: session default

07:25:59 Started executing query at Line 26
BEGIN

07:25:59 Started executing query at Line 28
WARNING: Blocked: Voter 1 has already voted in Election 1!
DO

Total execution time: 00:00:00.006

Figure 9: Duplicate vote blocked by the UNIQUE constraint (transaction.sql)

8. Execution & Screenshots

The scripts schema.sql, seed.sql, query.sql and transaction.sql are written in VS Code and run on a local PostgreSQL server in the order shown in Figure 10.

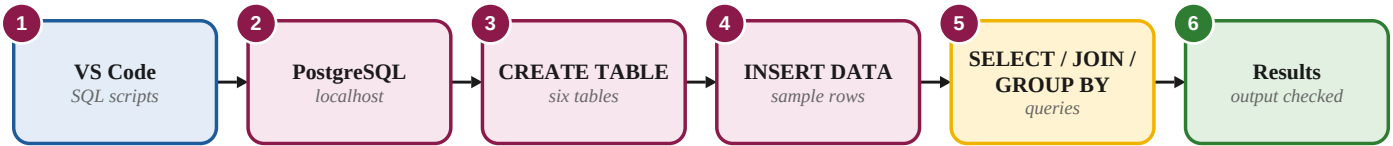


Figure 10: Execution flow of the VoteSecure SQL scripts

Step	Test	Expected result	Evidence
1	Create the tables	Tables exist with keys and constraints	Fig. 11, 12
2	Insert sample data (parents first)	Seeded rows are returned by the queries	Fig. 13
3	JOIN + GROUP BY + COUNT	Related rows combined; votes counted per candidate	Fig. 13
4	Winner query	One winner per constituency	Fig. 14
5	Second vote, same voter and election	Insert rejected (unique_violation caught)	Fig. 9

8.1 Screenshots

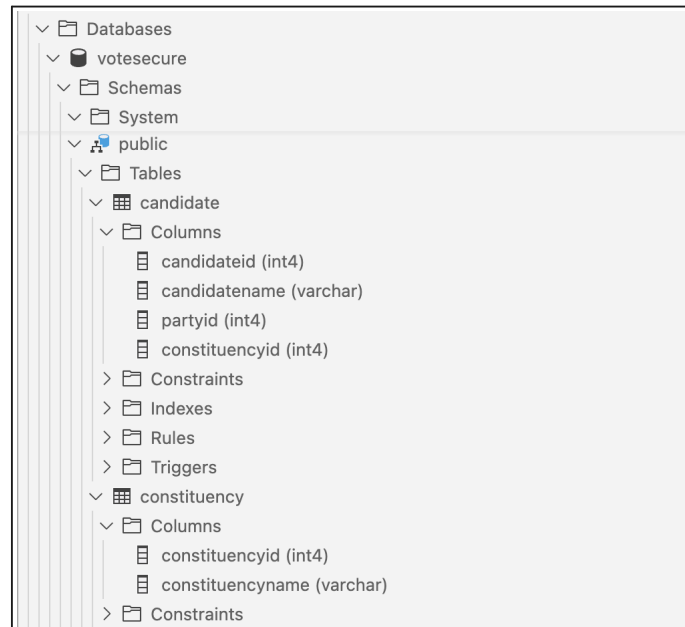


Figure 11: Database votesecure and its tables in the VS Code explorer

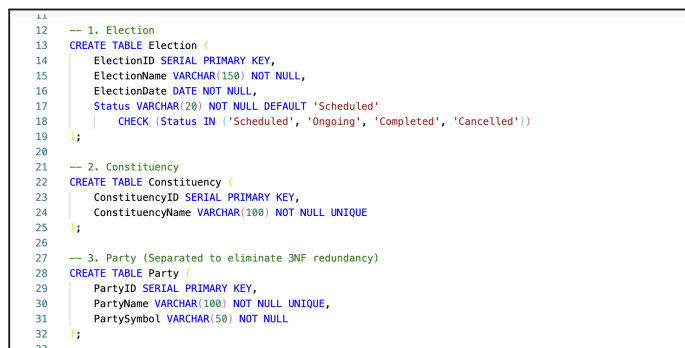


Figure 12: schema.sql – CREATE TABLE script with constraints

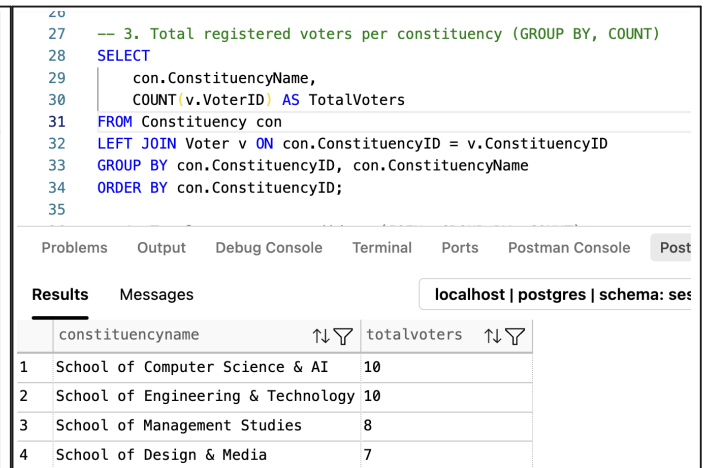


Figure 13: JOIN + GROUP BY + COUNT – registered voters per constituency

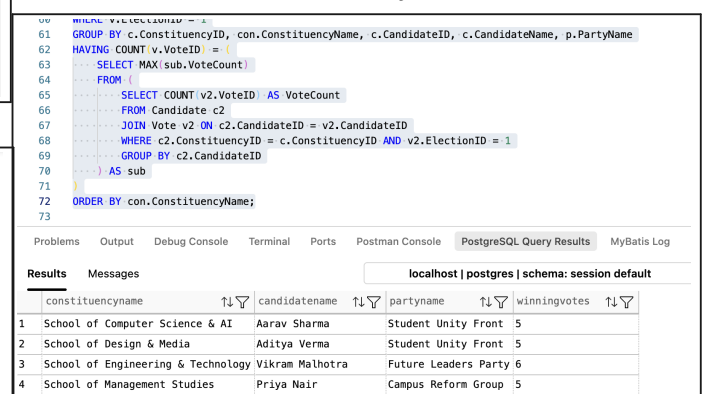


Figure 14: Winner query output – one winner per constituency (Election 1)

9. Results & Charts

9.1 Results / Observations

Aspect	Result	Evidence
Database creation	Tables created with primary keys, foreign keys and constraints (SERIAL IDs, NOT NULL, UNIQUE, CHECK, DEFAULT)	Fig. 11, 12
Sample data insertion	Seeded voters, candidates and parties are returned by the queries	Fig. 13
JOIN queries	Voters joined to constituencies; candidates joined to parties, constituencies and votes	Fig. 13, 14
Vote counting	GROUP BY with COUNT gives 10, 10, 8 and 7 registered voters per constituency	Fig. 13
Winner determination	MAX subquery returns exactly one winner for each of the four constituencies in Election 1	Fig. 14
Duplicate voting prevention	A second vote by Voter 1 in Election 1 is blocked by UNIQUE (VoterID, ElectionID)	Fig. 9
Referential integrity	Every vote must refer to an existing voter, candidate, constituency and election	Fig. 3, §4

9.2 Election 1 Winners (from the query output in Figure 14)

Constituency	Winner	Party	Votes
School of Computer Science & AI	Aarav Sharma	Student Unity Front	5
School of Design & Media	Aditya Verma	Student Unity Front	5
School of Engineering & Technology	Vikram Malhotra	Future Leaders Party	6
School of Management Studies	Priya Nair	Campus Reform Group	5

9.3 Result Charts

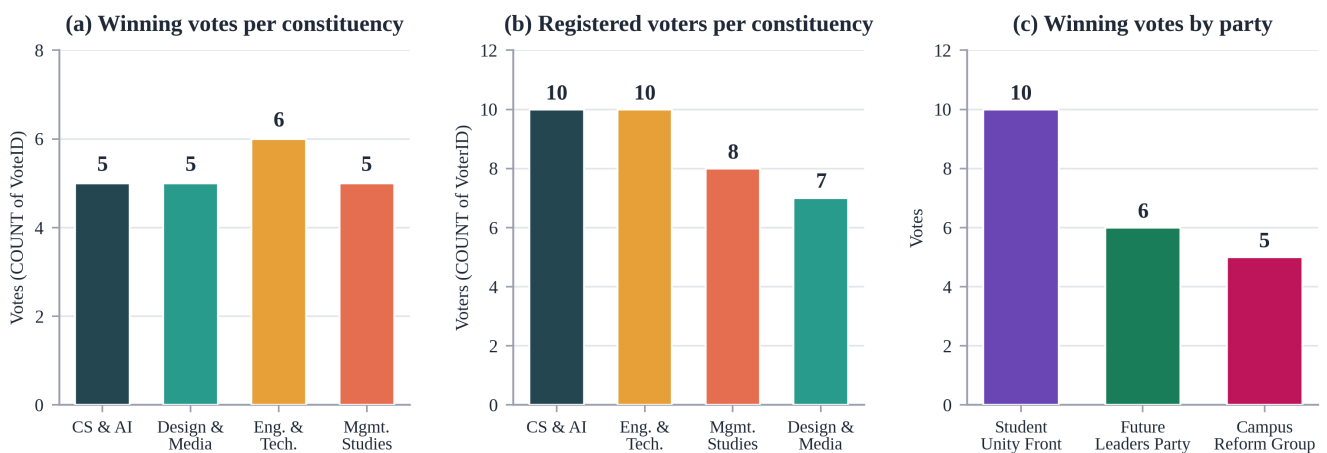


Figure 15: (a) winning votes per constituency and (c) winning votes by party, from Figure 14; (b) registered voters per constituency, from Figure 13

9.4 Observations

- Winners, vote counts and voter totals are all **derived by queries**; nothing is typed by hand.
- Each constituency returned a single winner; Vikram Malhotra led his constituency with 6 votes, and the other three winners polled 5 votes each.
- Student Unity Front won two of the four constituencies; Future Leaders Party and Campus Reform Group won one each.

10. Real-World Application & Future Scope

10.1 Real-World Applications

The same design (voters, candidates, parties, constituencies, elections, and one vote per voter per election) suits any small, rule-based election:

Setting	How the design applies
College elections	Class representatives or department heads, with departments as constituencies
University elections	Student union elections across schools or campuses
Student councils	Annual council elections with one vote per student
Clubs	Selecting club office bearers from registered members
Organizations	Committee or board elections among registered members

10.2 Future Scope

VoteSecure can grow from a SQL-based database into a complete election platform (Figure 16). Only the first box is implemented today.

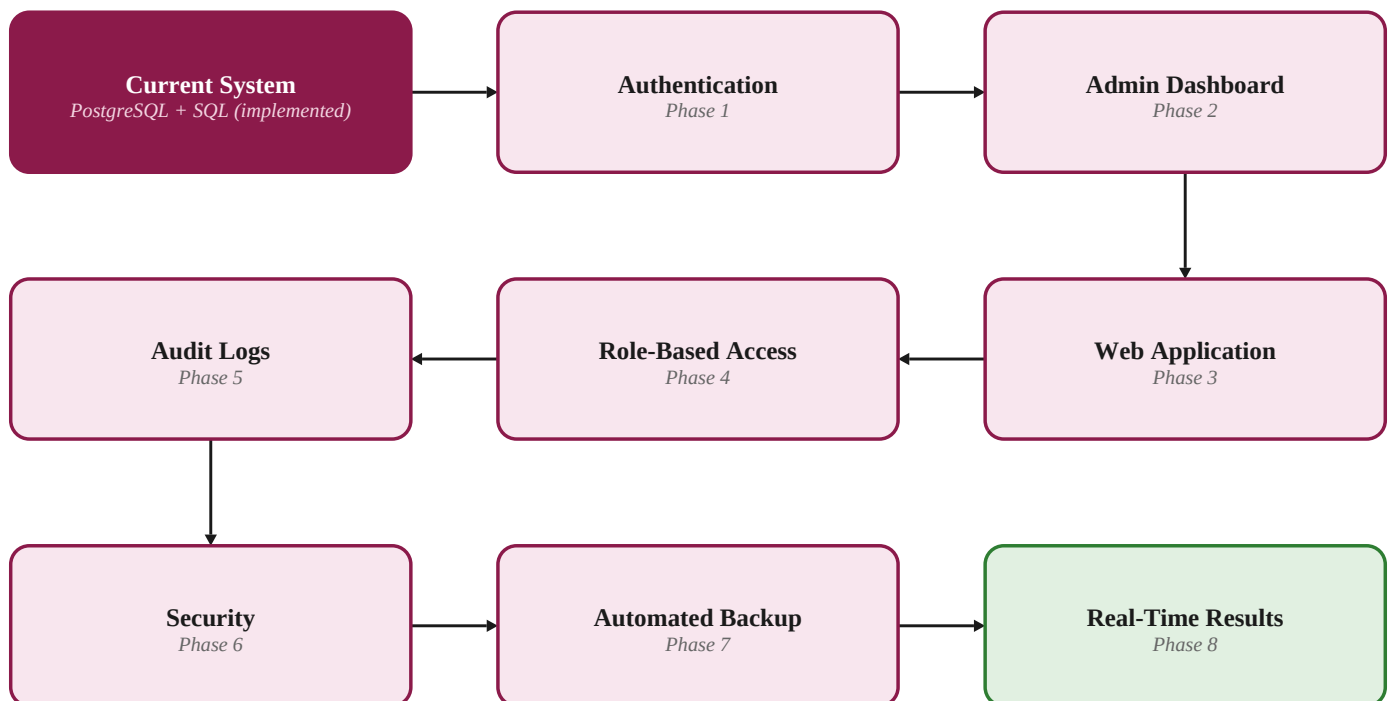


Figure 16: Future enhancement roadmap (all phases after the current system are planned, not implemented)

Current (implemented)	Future (planned, not implemented)
PostgreSQL database with six related tables	Authentication, role-based access and an admin dashboard
DDL, DML and DQL scripts with sample data	Web application with real-time results
Constraints: PK, FK, UNIQUE, NOT NULL, CHECK	Audit logs, security hardening and automated backup
JOIN, GROUP BY, aggregate and MAX-subquery reports	Scaling from one election to many elections and campuses

11. Conclusion

VoteSecure shows how a real election process can be modelled and managed with a relational database. The project covers relational database design, the ER model, normalization, SQL, keys, constraints, JOINS, GROUP BY, aggregate functions, a MAX subquery, duplicate voting prevention and winner determination. Every voter is stored once, every vote is linked to a candidate, a constituency and an election, duplicate voting is blocked by the database itself, and constituency winners are calculated with SQL the moment polling closes.

11.1 Concepts Demonstrated

Concept	Demonstrated by	Concept	Demonstrated by
Relational design	Six related tables	JOINS	Multi-table result queries
ER model	Six entities, 1 : N links, Vote as junction	GROUP BY and aggregates	COUNT, SUM, AVG, MIN, MAX
Normalization	Party separated from Candidate	MAX subquery	Highest count per constituency
SQL	DDL, DML and DQL scripts	Duplicate-vote prevention	UNIQUE (VoterID, ElectionID)
Keys	Primary and foreign keys in every table	Winner determination	JOIN + GROUP BY + COUNT + MAX
Constraints	NOT NULL, UNIQUE, CHECK, DEFAULT	Referential integrity	FOREIGN KEY on Vote

11.2 Case Study Deliverables

Deliverable	Where in this report
ER Diagram – entities, attributes, relationships	Section 3, Figure 3
Schema design – normalized tables with keys and constraints	Sections 4 and 5
SQL scripts – DDL for tables, DML for sample data	Section 5 (schema.sql, seed.sql)
Query set – JOINS, subqueries, aggregate functions	Sections 6 and 7 (query.sql)
Constraint enforcement – duplicate voting prevented	Sections 5 and 7 (transaction.sql), Figure 9
Working database with documented query outputs	Section 8, Figures 11–14

11.3 Learning Outcomes

- Translating a real-world process into an ER diagram and a relational schema with correct keys and cardinalities.
- Removing redundancy through normalization and enforcing business rules with constraints instead of application code.
- Using JOIN, GROUP BY, aggregate functions and a correlated MAX subquery to derive results directly from stored data.

11.4 Scope

- The project is database-only (PostgreSQL + SQL): no frontend, backend, authentication or user-role layer is implemented.
- Execution was carried out on a local PostgreSQL server through VS Code; the screenshots in Section 8 show the actual output.